Edge vs Cloud: How Do We Balance Cost, Latency, and Quality for Large Language Models Over 5G Networks?

Minsu Kim[1], Pinyarash Pinyoanuntapong[2], Bongho Kim[2], Walid Saad[1], and Doru Calin[2]

[1]Bradley Department of Electrical and Computer Engineering, Virginia Tech, Arlington, VA, USA.
[2]MediaTek, Warren, NJ, USA.

2025 IEEE Wireless Communications and Networking Conference (WCNC) | 979-8-3503-6836-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/WCNC61545.2025.10978177

***Abstract*—Large language models (LLMs) can perform a plethora of tasks, however, they often require cloud servers for deployment due to their computing cost and size. Meanwhile, small and cost-effective LLMs can be deployed on edge devices (e.g., mobile devices), but they often exhibit lower response quality than larger models. In this paper, a measurement-driven training framework is proposed for a device-side artificial intelligence (AI)-enabled router that selects the best LLM in terms of cost, latency, and performance. In the considered framework, a mobile device uses its local LLM (sub-billion LLM), while having access to a bigger server LLM (GPT-4) hosted on a 5G core network. The mobile device has a router that sends an input prompt to the local or server LLM to optimize the cost, latency, and performance of output LLM responses. To train the router, the dataset is constructed by measuring the cost, latency, and performance of the server LLM and the local LLM on a mobile device. The structural causal model (SCM) of the measured dataset is identified. To further improve performance, a causality-driven data augmentation method is also proposed based on the discovered SCM. Real-world experimental results show that the proposed framework can improve the cost and latency by 50% and 31%, respectively, with only a 2.13% performance loss compared to a baseline that only uses the server LLM.**

I. Introduction

Large language models (LLMs) are capable of executing a broad range of tasks including question-answering, code generation, and open-ended conversations [1]. These recent advances in LLMs have led to several real-world deployments such as ChatGPT [2] and Apple Intelligence [3]. To use LLMs, clients interact with various available language models on their mobile devices [3] or cloud servers [2]. However, LLMs vary the computing costs and model sizes. For instance, the QWEN2 models [4] range from sub-billion 0.5B to 72B parameters, and OpenAI's GPT-4 is rumored to be larger than one trillion parameters [5]. LLMs are even different with respect to the data that they are trained on. These factors lead to wide variations in the cost, latency, and performance.

Big LLMs (e.g., Llama 3.1: 405B, GPT-4) tend to work well for complex tasks, but they are typically deployed by cloud servers due to the high computing and memory requirements. Meanwhile, small LLMs (e.g., QWEN2:0.5B, flan-t5: 0.8B) are generally cost-efficient and can be run on mobile devices. Moreover, they can also provide privacy benefits because prompts are not transmitted to servers. However, smaller models tend to perform worse in terms of response quality when handling complex tasks. This tradeoff presents a challenge for the practical deployment of LLMs in wireless networks. One can process all client prompts on its mobile device, thereby saving costs by 80% (e.g., QWEN2:0.5B vs GPT-4 as shown in Fig. 6a). However, using only small LLMs can result in low-quality responses. Although a client can transmit all prompts to big LLMs at cloud servers, the computing cost and latency will be prohibitive. Moreover, the communication latency will be also non-negligible due to the uncertainty in channel and core network conditions.

To solve this tradeoff, LLM routing can be a promising solution. On a mobile device, an artificial intelligence (AI)-enabled router, hereinafter called 'router', first processes an input prompt, then it decides which LLM to use. The router predicts which LLM can optimize the cost, latency, and quality of output response for input prompts. For instance, if a small LLM on a mobile device can provide a response similar in quality to a big LLM on a server, the router chooses the small LLM to process the input. Similar examples have been also deployed in Apple's iPhone AI features [3]. However, LLM routing is challenging because a router needs to predict the cost, latency, and performance of multiple LLMs for an input prompt before the actual LLM inference. Prompts are highly dynamic with respect to lengths, contents, and complexities. Hence, the router needs to understand the capability of each LLM. Moreover, the router needs to consider the hardware of the mobile device (e.g., battery level and computing resource) and the client's preference toward cost, latency, and performance when making a decision.

A few recent works [6]–[10] explored the tradeoff pertaining to the choice between multiple LLMs. In [6], the authors used multiple LLMs for a single prompt and chose the best response among them. The work in [7] used a cascade of LLMs by processing an input prompt sequentially until one of the LLMs meets a predefined confidence score. The authors in [8] used a small LLM to verify the response of another small model before routing to a bigger LLM. However, these prior works [6]–[8] need to call more than one LLM for a single input, thereby incurring significant computing cost. Meanwhile, similar to our work, in [9] and [10], LLM routing is considered between small and big LLMs by selecting only one optimal model based on the performance. However, [9] and [10] did

W. Saad was supported by the U.S. National Science Foundation under Grant CNS-2114267.

979-8-3503-6836-9/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Gebze Technical University. Downloaded on November 10,2026 at 06:25:18 UTC from IEEE Xplore. Restrictions apply.

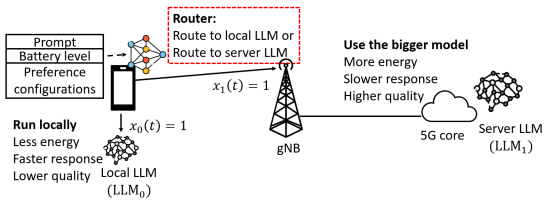


Fig. 1: An illustration of the LLM routing framework over a 5G network.

not consider the computing cost and latency of using LLMs on a mobile device and server. Moreover, the end-to-end (E2E) communication latency between a mobile device and server was not also considered in prior works [6]–[10] despite it being non-negligible due to channel and core network conditions.

The main contribution of this paper is a novel *measurement-driven* training framework for a router that can choose the best LLM by considering (a) the cost, latency, and performance of LLMs, (b) the client's preference configuration toward each term in (a), and (c) the battery level of the client's mobile device. In our framework, a mobile client has a local LLM (sub-billon LLM), while having access to a bigger LLM (GPT-4) hosted at the core network of a 5G system. An AI-enabled router is deployed on the mobile device, and it decides which LLM to use to process a client's input prompt. The local LLM is small and cost-efficient, but the response quality is generally low. Meanwhile, the server LLM can handle various tasks with higher qualities, but its computing cost is expensive, and the latency is larger than the local LLM due to the size and communication latency. Moreover, the router also needs to consider the client's preference toward the cost, latency, and LLM performance, as well as the battery level of the device when making a decision. Since this tradeoff cannot be analytically modeled, we propose a *measurement-driven* training framework for the router. We measure the cost, latency, and performance of the server LLM and local LLM on a mobile device to prepare the training dataset over various real-world prompts. We then identify a structual causal model (SCM) for our measured dataset. Based on the discovered SCM, we propose a new data augmentation method inspired by causal distribution generalization to enhance the performance of the router. Real-world experimental results show that our framework can improve the cost and latency by 50% and 31%, respectively, with only a 2.13% performance loss compared to a baseline that only uses the server LLM.

The rest of this paper is organized as follows. Section II presents the proposed system model and problem formulation. In Section III, we describe the measurement. Section IV explains the training data generation. Section V provides experiment results, and conclusions are drawn in Section VI.

II. System Model

We consider a mobile client that uses LLM inference services over a 5G network as shown in Fig. 1. We assume that the client has a local LLM on its mobile device, and a server hosts a bigger LLM (i.e., server LLM) on the 5G core network connected via the gNodeB (gNB). The client can access the server LLM via the supported API on the device. Due to the resource constraints at the mobile device, local LLMs are usually much smaller than the ones hosted by the server. Hence, bigger models can provide responses with better quality than local LLMs. However, the size of bigger LLMs often induces a large computing latency and cost (e.g., energy consumption) [11]. Moreover, the E2E latency between the client and server can be non-negligible due to channel and 5G core network conditions [12], [13]. Hence, a tradeoff exists between the cost, latency, and performance when deciding which LLM to use. To optimize this tradeoff, on the client side, we deploy an AI-enabled router that routes an input prompt to the local LLM or the server LLM. The router takes a prompt, the client battery level, and the client's configuration as an input. Here, the configuration is the client's preference toward the cost, latency, and performance of the LLMs. The router then predicts the cost, latency, and quality of the response of each LLM. Then, it routes the input to the chosen LLM.

A. Evaluation metrics

To design the decision process of the router, we first need metrics to evaluate the performance of each LLM.

1) Cost of LLM: The cost of the server LLM includes the communication computing costs. Meanwhile, the local LLM cost induces only a computing cost. The client accesses the server LLM by calling the API, and each API call costs a certain amount of credit according to model pricing lists [2]. As such, we use \$/token to capture the computing cost of the local and server LLMs. For the communication cost, we track the energy consumption of transmission and reception. Then, we convert the measured energy consumption to \$ usage.

2) Latency of LLM: The latency of the server LLM consists of communication latency and computing latency. Meanwhile, the latency of the local LLM only includes the computing latency. For the computing latency, we measure the time-to-first-token (TTFT) of LLMs. Specifically, TTFT is the elapsed time for an LLM to generate the first output token. Most LLM services send each output token to a user (i.e., streaming) rather than waiting to generate the entire output response. The communication latency is the E2E latency between the mobile device and the server for transmitting a prompt and receiving the first token.

3) Response quality of LLM: GPT-based evaluations have demonstrated a high correlation with human perception for measuring the quality of LLM responses [14]. However, these approaches have very high computational costs. Hence, we adopt the BART score [15] to measure the response quality of LLMs. The BART score is more computationally efficient since it uses a compact pre-trained BART model for the evaluation. Moreover, it also shows a high correlation with GPT-based methods on various prompts [6].

B. Problem formulation

We denote the local LLM and server LLM as LLM_0 and LLM_1, respectively. For an input prompt $I[n]$ with index n, the router chooses the local or server LLM to optimize the cost,

Authorized licensed use limited to: Gebze Technical University. Downloaded on November 10,2026 at 06:25:18 UTC from IEEE Xplore. Restrictions apply.

latency, and quality of the output response. Essentially, this is a binary selection problem, where the router outputs binary decision variables $x_m[n]$ such that $\sum_{m\in\{0,1\}} x_m[n] = 1$ for a given input prompt $I[n]$. If $x_0[n] = 1$, the router sends the input prompt to LLM_0, otherwise, it routes to LLM_1. More formally, the router's decision making process can be formulated as follows

$$\min_{X[0],...,X[N-1]} \frac{1}{N}\sum_{n=0}^{N-1}\sum_{m\in\{0,1\}} x_m[n]\big(\alpha E_m[I[n]] + \beta D_m[I[n]] + (1-\alpha-\beta)\frac{1}{A_m[I[n]]}\big) \quad (1)$$

$$\text{s.t.} \sum_{m\in\{0,1\}} x_m[n] = 1, \forall n \in \{0,...,N-1\}, \quad (2)$$

$$X[n] = \{x_0[n], x_1[n]\}, \forall n \in \{0,...,N-1\}, \quad (3)$$

where $E_m[I[n]]$ is the cost of using LLM_m for processing $I[n]$, $D_m[I[n]]$ is the latency to receive the first generated output token of LLM_m, and $A_m[I[n]]$ is the BART score of the response of LLM_m for $I[n]$. We denote (α, β) as the client's preference configuration toward the cost, latency, and BART score such that $0 \le \alpha, \beta \le 1$ and $0 \le \alpha + \beta \le 1$.

The above problem is challenging because there are no known analytical expressions of $E_m[I[n]], D_m[I[n]]$, and $A_m[I[n]]$ for the given input prompt $I[n]$. Unlike images that can have predefined resolutions and channels, prompts are highly dynamic with respect to the lengths and content. For two prompts with the same length, an LLM can provide totally different responses depending on the context of the prompts. Due to the LLM's auto-regressive decoding process, it is also challenging to analytically model how many output tokens will be generated for $I[n]$.

Our approach is to train an AI-enabled router that takes $I[n]$ as an input and can predict which LLM can optimize (1). Due to the analytical intractability of (1), we propose a *measurement-driven* training framework where we measure $E_m[I[n]], D_m[I[n]]$, and $A_m[I[n]$ with vast and dynamic input prompts on a mobile device and a server.

III. LLM Metric Measurements

We use a proprietary mobile device with 16GB of memory with the MediaTek Dimensity 9300 chipset. A local LLM should be compact enough to fit into the limited memory. Hence, we use the state-of-the-art sub-billion LLM model QWEN2:0.5B [4] for the local LLM. To capture the strong performance and computing requirements of a server model, we use the GPT4-1106-preview with the OpenAI API. To measure the performance of the LLMs across various tasks, we use the MixInstruct dataset [6], which is a large-scale collection of real-world instruction-following prompts such as question answering and summarization.

A. Measuring $D_m[I[n]$

$D_0[I[n]$ consists of only computing latency, and $D_1[I[n]$ includes both computing and communication latency. As mentioned in Section II-A2, for the computing latency, we measure the TTFT of each $I[n]$ for both the local and server LLM over the 9K prompts from the MixInstruct dataset. Hence, we have $D_0[I[n] = \text{TTFT}_0[I[n]]$, where $\text{TTFT}_0[I[n]]$ is the TTFT of $I[n]$ with the local LLM. For the communication latency between the mobile device and the server, we use a 5G measurement dataset from [16]. We assume a 5G non-standalone (NSA) campus network with the n78 band from 3.7 GHz to 3.8 GHz with the 100 MHz of bandwidth. We use the measurement data of the uplink and downlink E2E latencies between the mobile device and the 5G core with a packet size of 512 bytes over TCP connections [16]. Note that prompts consist of texts, and each character usually takes 1-4 bytes with the UTF-8 encoding. Hence, most of the prompts of the MixInstruct dataset can be transmitted in a single packet. We define D_{ul} and D_{dl} as the E2E uplink and downlink latency, respectively. Based on the measurement data in [16], we use a Gaussian distribution to model D_{ul} and D_{dl} i.e., $D_{\text{ul}} \sim \mathcal{N}(0.015, 0.009)$ and $D_{\text{dl}} \sim \mathcal{N}(0.013, 0.004)$, respectively. Here, the 99% percentiles of upload and download E2E latency are 22 ms and 20 ms, respectively. This also aligns well with the Ericsson E2E latency measurement reports [17]. Hence, with the computing latency, we have $D_1[I[n]] = \text{TTFT}_1[I[n]] + D_{\text{ul}} + D_{\text{dl}}$.

B. Measuring $E_m[I[n]$

We measure the cost of the local LLM $E_o[I[n]]$ on the mobile device using prompts from the MixInstruct dataset. As mentioned in Section II-A1, we adopt '$/token' for the cost. We derive the '$/token' of the local LLM by tracking battery usage for running the local LLM. Since a battery is essentially a storage device for energy, we use the electricity cost to convert the 'battery usage per token' into '$/token'.

To track the battery in a fine-grained scale, we developed an Android App that can measure the battery level per second. For consistency, we keep the same brightness of the display during the measurement and remove all background apps. We also measured the idle state battery consumption to derive the pure computing cost.

We measured the average time t_{LLM_0} to drain 1% of the battery for running the local LLM. Then, for the battery capacity B_{capacity} [mAh] and the battery voltage V_B [mV] of the phone, we have the local LLM power consumption [Joule/sec] as follows

$$E^{\text{time}}_{\text{LLM}_0} = 3.6 \times B_{\text{capacity}} \times t^{-1}_{\text{LLM}_0} \times V_B, \quad (4)$$

where B_{capacity}= 1000, V_B = 3.7, and t_{LLM_0} = 81.37. We also tracked the average number of output tokens per second during the measurement. Then, we can derive the energy consumption per token [Joule/token] as $E^{\text{token}}_{\text{LLM}_0} = E^{\text{time}}_{\text{LLM}_0}/T_{\text{out}}$, where T_{out} is [# of tokens/sec]. We finally derive '$/token' using the electricity cost NJ_{elec} [$/kWh] of New Jersey, USA, as follows

$$E_0[I[n]] = \text{NJ}_{\text{elec}} \times E^{\text{token}}_{\text{LLM}_0} \times 3.6 \times 10^{-5} \times N^0_{\text{out}}[I[n]], \quad (5)$$

where $\text{NJ}_{\text{elec}} = 0.17$, $N^0_{\text{out}}[I[n]]$ is the number of generated output tokens of the input $I[n]$ with LLM_0.

On the server side, the cost $E_1[I[n]]$ consists of the computing and communication costs. For the computing cost, we refer

Authorized licensed use limited to: Gebze Technical University. Downloaded on November 10,2026 at 06:25:18 UTC from IEEE Xplore. Restrictions apply.

Prompt	α	β	$\mathbf{WS_0}$	$\mathbf{WS_1}$	Decision
$I[0]$	α_1	β_1	$\alpha_1 E_0[I[0]] + \beta_1 D_0[I[0]] + (1-\alpha_1-\beta_1)A_0[I[0]]$	$\alpha_1 E_1[I[0]] + \beta_1 D_1[I[0]] + (1-\alpha_1-\beta_1)A_1[I[0]]$	$x_0[0]=1, x_1[0]=0$
$I[1]$	α_1	β_1	$\alpha_1 E_0[I[1]] + \beta_1 D_0[I[1]] + (1-\alpha_1-\beta_1)A_0[I[1]]$	$\alpha_1 E_1[I[1]] + \beta_1 D_1[I[1]] + (1-\alpha_1-\beta_1)A_1[I[1]]$	$x_0[1]=0, x_1[1]=1$
$I[2]$	α_1	β_1	$\alpha_1 E_0[I[2]] + \beta_1 D_0[I[2]] + (1-\alpha_1-\beta_1)A_0[I[2]]$	$\alpha_1 E_1[I[2]] + \beta_1 D_1[I[2]] + (1-\alpha_1-\beta_1)A_1[I[2]]$	$x_0[2]=0, x_1[2]=1$

Fig. 2: An illustration of our training dataset.

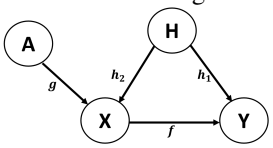


Fig. 3: An illustration of the SCM model M of our dataset.

to OpenAI pricing [2], and the current GPT4-1106-preview costs \$10 per one million tokens. For the communication cost, we use experimental data from [18], which measured the power consumption of transmission/reception of 5G smartphones using 5G NR with n78 band for 100 MHz bandwidth. According to [18], the transmission and receive power of browser-based applications can be given by $P_{\text{tx}} = 2$ [W] and $P_{\text{rx}} = 1$ [W], respectively. Then, we have $E_1[I[n]] = 10^{-7} \times N^1_{\text{out}}[I[n]] + \text{NJ}_{\text{elec}}(D_{\text{ul}}P_{\text{tx}} + D_{\text{dl}}P_{\text{rx}}) \times 3.6 \times 10^{-5}$.

C. Measuring $A_m[I[n]$

We use a pre-trained BART model from [15] to measure the BART scores of output responses of the LLMs. For the output response $O_m[n]$, we have the following BART score:

$$A_m[I[n]] = \sum_{k=0}^{K-1} \omega_k \log p(O^k_m[n] | O^{<k}_m[n], J[n], \text{LLM}_m), \quad (6)$$

where K is the number of output tokens, ω_k is the weight for token k, and $J[n]$ is the target text.

IV. Dataset Generation

With the measured cost, latency, and BART score data in Section III, we now generate training dataset for the router. Our dataset is shown in Fig. 2. The inputs are 'Prompt, α, and β', and the outputs are the weighted sums in WS_0 and WS_1 columns and ground-truth decisions. Here, the configuration (α, β) is a part of the input because it represents the preference of the client toward each term in (1). The router takes a given prompt and (α, β), and predicts the weighted sum of cost, latency, and BART score of each LLM. Then, it selects an LLM with a smaller weighted sum to optimize (1).

However, we found that 9K prompts are insufficient to train the router. Due to the complexity of analyzing the content of $I[n]$, it is challenging to capture how $I[n]$ and (α, β) lead to a certain weighted sum of predicted cost, latency, and BART score. Hence, from [19], [20], we propose data augmentation inspired by causal distribution generalization.

We first introduce the structural causal model (SCM) M of our dataset as shown in Fig. 3. We consider a response Y, covariates X, unobserved variables H, and exogenous variables A, which come from the outside of the model. Here, Y can be considered as a decision of the router, X is the weighted sums of cost, latency, and BART score, and A is (α, β) that come from the client. We define f as a causal function and P_M as the joint distribution over the observed variables (X, Y, A). An SCM $M \in \mathcal{M}$ can be modified by interventions, which remove all incoming edges of covariates or exogenous variables and set specific values to them [19]. Here, we consider a set of interventions $\mathcal{I}$ on the exogenous variables (α, β). We denote $M(i)$ as an intervened SCM by intervention $i \in \mathcal{I}$. By performing interventions on (α, β), we can see the impact of different (α, β) on the router's decisions.

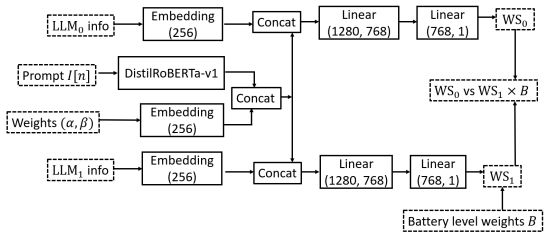


Fig. 4: An illustration of the router architecture.

We next provide a key lemma for the main result. We denote supp(X) as the support of random variable.

Lemma 1. *$\forall \tilde{M} \in \mathcal{M}$ with $P_{\tilde{M}} = P_M$, we have: $\tilde{f}(x) = f(x)$ and $\tilde{g}(a) = g(a)$, $\forall x \in \text{supp}(X)$ and $\forall a \in \text{supp}(A) \cup \text{supp}_I(A)$.*

Proof. For any two different our SCM models, how the causal function f works on $x \in \text{supp}(X)$ does not change. This is because f just compares which weighted sum is larger/smaller than the other. Similarly, in our SCM model, the function g outputs a weighted sum of cost, latency, and BART score for given (α, β). Hence, $\tilde{g}(a) = g(a)$ holds for all $a \in \text{supp}(A) \cup \text{supp}_I(A)$. □

Then, we have the following proposition [19]

Proposition 1. *Given Lemma 1 and $\text{supp}_{\mathcal{I}}(X) \subseteq \text{supp}(X)$, then, $(P_{\mathcal{M}}, \mathcal{M})$ admits generalization to $\mathcal{I}$: $\forall \epsilon > 0, \exists f^*_\epsilon$ such that $\forall \tilde{M} \in \mathcal{M}$ with $P_{\tilde{M}} = P_M$, it holds that*

$$\left| \sup_{i \in \mathcal{I}} \mathbb{E}_{\tilde{M}(i)}[Y - f^*_\epsilon(X)]^2 - \inf_f \sup_{i \in \mathcal{I}} \mathbb{E}_{\tilde{M}(i)}[Y - f(X)]^2 \right| \leq \epsilon. \quad (7)$$

Proposition 1 means that there exists an approximate solution f^*_ϵ whose distance to the optimal solution can be bounded by ϵ for any interventions. To find such a solution, we perform multiple interventions on (α, β), thereby having an augmented dataset for each (α, β). Specifically, each (α, β) leads to different weighted sums of the LLMs as shown in Fig. 2. By having multiple different datasets from intervening (α, β), the router can learn the impact of (α, β) on the ground-truth decisions more effectively.

V. Experimental Results

A. Router model

The router model is shown in Fig. 4. We use a pre-trained sentence-transformer model DistilRoBERTa-v1 [21] to embed

Authorized licensed use limited to: Gebze Technical University. Downloaded on November 10,2026 at 06:25:18 UTC from IEEE Xplore. Restrictions apply.

	Case 1	Case 2	Case 3
Accuracy	**94.38±1.02%**	82.27±0.84%	72.13 ±0.95%

TABLE I: Accuracy of the trained router.

prompts $I[n]$. The DistilRoBERTa-v1 maps an input sentence to a 768-dimensional embedding space. We embed the values of (α, β) using an embedding layer. To distinguish the local and server LLM, we also embed their information in a separate embedding layer as shown in Fig. 4. These embedded values are concatenated and fed into two separate linear layers. Each final layer outputs a predicted weighted sum denoted as WS_m for the corresponding LLM.

B. Training setup

We set $\alpha = \{0, 0.1, 0.2, ..., 1\}$ and $\beta = (1-\alpha)/5$ assuming the client has a strong preference toward maximizing the BART score. We then have 11×9K datasets from each (α, β). We use an ADAM optimizer with a learning rate of 0.001, batch size of 32, and 10 epochs. We divide 80% of the total dataset including augmented data into training and 20% as the test dataset. We use the mean square error loss function between WS_m from the router and the ground-truth value. We average all results over 5 random seeds.

C. Inference with the battery level

After the training, the router takes a prompt and (α, β) from the client and sends it to a better LLM. In Fig. 5, we observe that, as α increases, the router sends more prompts to the local LLM. Although this operates in alignment with the preference of the client, it does not consider the current battery level of the device. Indeed, during our measurements, we observed that LLM inference could drain a device's battery significantly. Hence, the battery can be completely drained if the router sends too many prompts to the local LLM when the device has a low battery level. Hence, we define a weight B to capture the battery level such that $B \in \{\text{High} : 1, \text{Mid} : 0.5, \text{Low} : 0.25\}$, where 'High', 'Mid', and 'Low' correspond to 100-70%, 69-30%, 30-0% battery levels, respectively. As shown in Fig. 4, B is multiplied with the WS_1 of the server LLM. Hence, for a low battery level, B decreases WS_1 so that the decision criteria for the server can be relaxed. We only consider three battery levels [High, Mid, Low], however, future work can investigate a more realistic battery level model.

D. Results

We first present the test accuracy of the trained router in Table. I. Case 1 is our default setting with the data augmentation using $\alpha = \{0, 0.1, 0.2, ..., 1\}$ and $\beta = (1-\alpha)/5$ for the training. Case 2 also used the data augmentation, but limited diversity with $\alpha = \{0, 1/11 \times 0.5, 2/11 \times 0.5, ..., 0.5\}$ and $\beta = (1-\alpha)/5$. Case 3 is without the data augmentation using a single $(\alpha = 0.1, \beta = 0.9)$. We can see that Case 1 outperforms the other cases. This corroborates our analysis in Proposition 1. Our SCM model admits generalization in interventions on (α, β), and training with multiple augmented datasets can improve the accuracy. Although Case 2 is better than Case 3, the considered interventions do not cover enough configurations, leading to a limited improvement.

In Fig. 5, we present the distribution of the processed prompts by the client and the server. As α increases, the preference toward minimizing the cost becomes more dominant than that of latency and BART score. As such, more prompts are processed locally to minimize the cost. As the battery level decreases, the decision criteria for the server becomes more relaxed. Hence, we can see that more prompts are processed by the server.

In Fig. 6, we present the average cost, latency, and BART score of the trained router. To the best of our knowledge, there is no prior work that optimized LLM selection between a mobile device and server in 5G networks. Hence, we consider three meaningful baselines: 1) ***random*** sends prompts randomly, 2) ***client-only*** uses only the local LLM, and 3) ***server-only*** uses only the server LLM.

From Fig. 5, it is clear that, as α increases, the router sends more prompts to the local LLM to save costs. We can also find configurations (α, β) that can balance the tradeoff effectively. For the 'High' battery level with $\alpha = 0.1$, Fig. 5a shows that 44% of the prompts were processed locally. However, there is only a 2.13% performance loss as shown in Fig. 6c compared to the *server-only* case. Meanwhile, the cost and latency are improved by 50% and 31%, respectively, compared to the *server-only* scenario. Similarly, for the 'Mid' battery level with $\alpha = 0.5$, 37% of the prompts can be processed locally with only 3.52% of the performance loss compared to the *server-only* scenario as shown in Figs. 6c and 5b. The cost and latency can be improved by 40% and 18.5%, respectively, compared to the *server-only* case. Therefore, the router can distinguish which prompts can be processed by the local LLM with minimal performance loss based on the analyzed context of a prompt. However, when the battery level is 'Low', most of the prompts are processed by the server unless α is big enough. For $\alpha = 0.8$, only 18% of the prompts were processed locally with 2.8% of the performance loss. Since α is much larger than the weight of the performance, the router becomes more conservative when predicting the response quality.

We also report the overhead of the router by measuring the inference latency of the trained router with the test datasets. The average inference latency is 8 ms. This overhead is negligible compared to the computing latency of the local and server LLM as shown in Fig. 6b.

VI. Conclusion

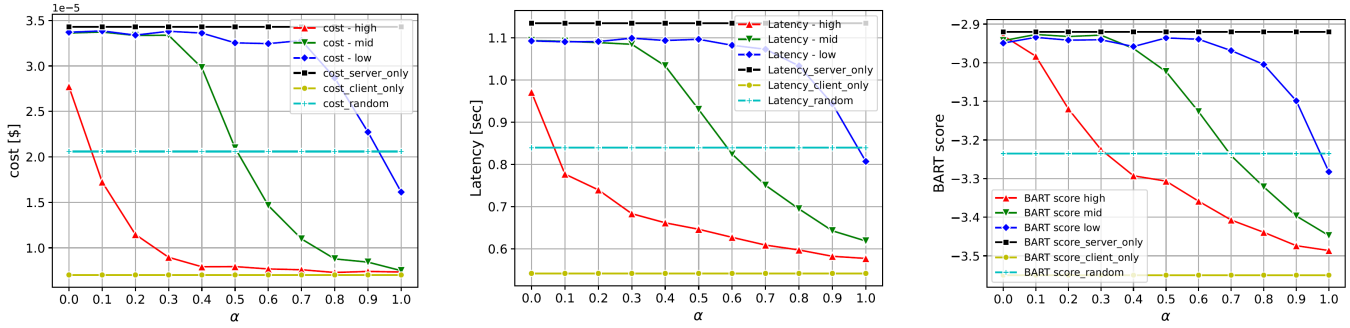
In this paper, we have studied an LLM selection problem between local and server LLMs to optimize the cost, latency, and performance of output LLM responses. To solve the problem, we have proposed an AI-enabled router on a mobile device and proposed a measurement-driven training framework. We have prepared a dataset by measuring the actual cost, latency, and performance of the local LLM on a mobile device and the server LLM. We also have proposed a data augmentation method inspired by causal distribution generalization to improve the

Authorized licensed use limited to: Gebze Technical University. Downloaded on November 10,2026 at 06:25:18 UTC from IEEE Xplore. Restrictions apply.



(a) High battery level (b) Mid battery level (c) Low battery level

Fig. 5: Distribution of the processed prompts by the client and the server.



(a) Cost (b) Latency (c) Bart score

Fig. 6: Average cost, latency, and BART score of the trained router.

accuracy of the router. Experiment results have shown that the trained router can achieve significant cost and latency improvements while maintaining output response quality. We also have shown the effectiveness of the proposed data augmentation for improving the accuracy of the router.

REFERENCES

[1] W. Saad, O. Hashash, C. K. Thomas, C. Chaccour, M. Debbah *et al.*, "Artificial general intelligence (AGI)-native wireless systems: A journey beyond 6G," *arXiv preprint arXiv:2405.02336*, 2024.

[2] OpenAI, "Openai inference api," https://openai.com/api/pricing/, Sep 2024.

[3] Apple, "Apple intelligence," https://www.apple.com/apple-intelligence/, Sep 2024.

[4] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou *et al.*, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.

[5] J. A. Baktash and M. Dawodi, "Gpt-4: A review on advancements and opportunities in natural language processing," *arXiv preprint arXiv:2305.03195*, 2023.

[6] D. Jiang, X. Ren, and B. Y. Lin, "Llm-blender: Ensembling large language models with pairwise ranking and generative fusion," in *Proc. of the Association for Computational Linguistics (ACL)*., Toronto, Canada, Jul 2023, pp. 14 165–14 178.

[7] L. Chen, M. Zaharia, and J. Zou, "Frugalgpt: How to use large language models while reducing cost and improving performance," *arXiv preprint arXiv:2305.05176*, 2023.

[8] A. Madaan, P. Aggarwal, A. Anand, S. P. Potharaju, S. Mishra *et al.*, "Automix: Automatically mixing language models," *arXiv preprint arXiv:2310.12963*, 2023.

[9] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee *et al.*, "Hybrid llm: Cost-efficient and quality-aware query routing," in *Proc. of International Conference on Learning Representations (ICLR)*, May 2024.

[10] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez *et al.*, "Routellm: Learning to route llms with preference data," *arXiv preprint arXiv:2406.18665*, 2024.

[11] M. Kim and W. Saad, *Toward Green Federated Learning*. Springer International Publishing, 2025.

[12] I. Parvez, A. Rahmati, I. Guvenc, A. I. Sarwat, and H. Dai, "A survey on low latency towards 5G: Ran, core network and caching solutions," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 4, pp. 3098–3130, May 2018.

[13] Y. Zhang, W. Cheng, and W. Zhang, "Multiple access integrated adaptive finite blocklength for ultra-low delay in 6G wireless networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 3, pp. 1670–1683, Mar 2024.

[14] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in Neural Information Processing Systems*, vol. 36, pp. 46 595–46 623, Dec 2023.

[15] W. Yuan, G. Neubig, and P. Liu, "Bartscore: Evaluating generated text as text generation," *Advances in Neural Information Processing Systems*, vol. 34, pp. 27 263–27 277, 2021.

[16] J. Rischke, P. Sossalla, S. Itting, F. H. Fitzek, and M. Reisslein, "5G campus networks: A first measurement study," *IEEE Access*, vol. 9, pp. 121 786–121 803, Aug 2021.

[17] R. Ludwig, "Who cares about latency in 5G," https://www.ericsson.com/en/blog/2022/8/who-cares-about-latency-in-5G, Aug 2022.

[18] D. Xu, A. Zhou, X. Zhang, G. Wang, X. Liu, C. An *et al.*, "Understanding operational 5G: A first measurement study on its coverage, performance and energy consumption," in *Proc. of ACM SIGCOMM*, Jul 2020, pp. 479–494.

[19] R. Christiansen, N. Pfister, M. E. Jakobsen, N. Gnecco, and J. Peters, "A causal framework for distribution generalization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 6614–6630, Oct 2021.

[20] C. K. Thomas, C. Chaccour, W. Saad, M. Debbah, and C. S. Hong, "Causal reasoning: Charting a revolutionary course for next-generation ai-native wireless networks," *IEEE Vehicular Technology Magazine*, vol. 19, no. 1, pp. 16–31, Feb 2024.

[21] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

Authorized licensed use limited to: Gebze Technical University. Downloaded on November 10,2026 at 06:25:18 UTC from IEEE Xplore. Restrictions apply.